MQTT

quarta-feira, 30 de abril de 2025 09:56

Parecido com o HTTP, mas foi projetado para a entrega de mensagens
Foi desenvolvido pela IBM em 1999

O HTTP é pesado quando se leva em consideração o IoT (Chega a 1.5kb), pois esses microcontroladores tem pouca capacidade para rodar esse HTTP

MQTT foi feito para trabalhar com microcontroladores, que possuem espaço (memória e armazenamento), processamento pequenos(taxa de 20b)

Com HTTP a quantidade de informação que pode ser enviada é muito reduzida, sobrecarregando os dispositivos com pouca coisa

MQTT foi criado inicialmente para monitorar oleodutos, via satélite, e caso utilizasse HTTP os pacotes teriam problemas de envio, pelo seu tamanho

Protocolo assíncrono -- Quando uma mensagem não precisa ser acessada imediatamente, o protocolo trabalha de forma que a resposta não precisa ser respondida no momento que foi realizado o pedido

Tem um intermediário (broker) que recebe a informação e o assinante que é quem consome essas mensagens

Tópicos

Toda mensagem criada pelo MQTT é enviada para o broker e essa mensagem será enviada, e o publisher envia a mensagem/informação para um broker que fica dentro dos tópicos

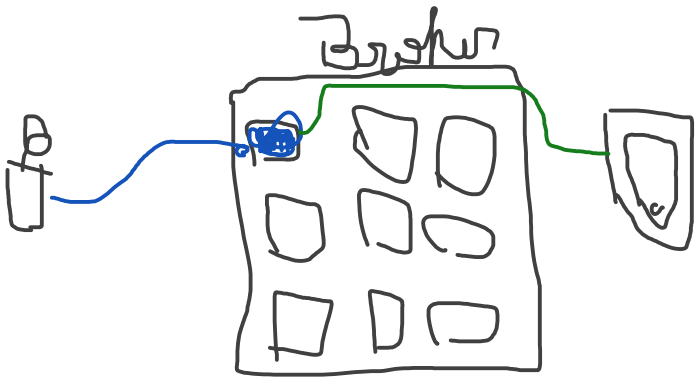
Para receber essas mensagens precisa ter acesso ao tópico (os subscribles recebem as mensagens que foram enviadas para os tópicos)

==========

Publisher – quem envia dados
Broker – armazena os dados
Subscriber – quem recebe os dados

Dentro dos brokers tem os tópicos:

- Quando o publisher publica algo, ela vai ser armazenada em algum dos tópicos dentro do broker, mas não em todos



O azul, é um sensor que envia (publisher) a informação para o tópico 1 do borker, e o verde representa o subscrible, que recebe a informação

Como um sensor de presença sendo ligado a uma luz

Dois sensores podem trabalhar em um mesmo tópico:

Sensor de presença, e sensor de luminosidade informam um mesmo tópico sobre ligar ou não a luz de um local
Mais de um assinante também pode acessar o mesmo tópico

Lâmpadas inteligentes mandam mensagem para os tópicos dizendo se ela está ligada ou desligada, e esse broker vai guardar a informação dela

==========

O tópico pode ser dividido em sub tópicos:

Tópico SENAI --> subtópicos: Sala 1 Sala 2 Sala 3 e dentro de cada sala poderia ter outros subtópicos como ar condicionado, luzes, tomadas

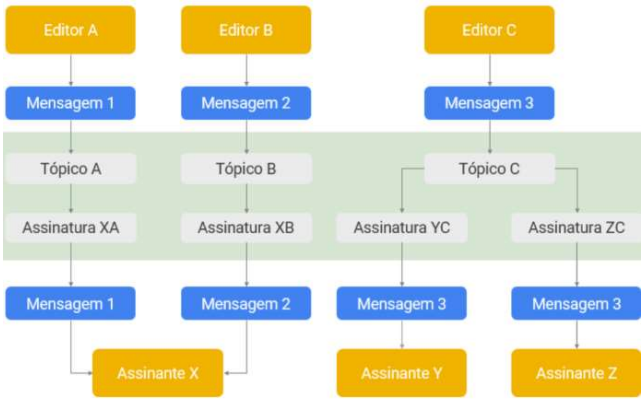
SENAI/SALA 1/AR CONDICIONADO

Ar condicionado teria o sensor de temperatura, controle das pás, modo de operação, estado (ligado ou desligado) potência de ventilação, portanto vários sensores seriam ligados a esse tópico, e todos eles são disponibilizados para o consumo dos assinantes

PWM

SENAI/+/ar condicionado --> Trata direto o ar condicionado
SENAI/# realiza a coleta geral dos dados (de todos os publishers)

==========

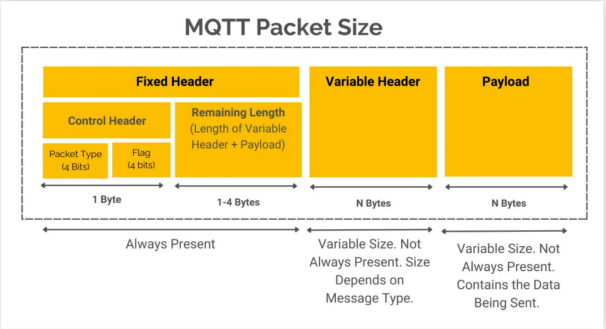


Assinante X pode ser um banco de dados que recebe todos os dados dos tópicos
Um tópico pode ter duas assinaturas, como no exemplo YC e ZC (um poderia ser o banco de dados e um poderia ser um aparelho de controle, ou seja, duas ações diferentes, um armazena e o outro controla)

Armazenar as informações recebidas é essencial pois a inteligência das coisas só pode funcionar caso essas informações sejam previamente dadas a ele

==========

MQTT é ideal pois os dados que trafegam são leves e não sobrecarregam os dispositivos
Promove o baixo custo de energia, pois consegue deixar dispositivos offline, sendo as mensagens assíncronas
QoS, persistência e reconexão automática (também pelas mensagens assíncronas)
Vários sensores podem conversar com o mesmo broker sem nenhum problema, assim como assinantes e publishers



O pacote tem 4 bits, e ele que define se será um publisher ou assinante

Remaining Length – temo tamanho real do cabeçario
Cabeçario variável – metadados, qual o tópico que preciso enviar a mensagem, quais os tópicos que vão receber, e são variados pois esses remetentes e suas quantidades mudam a todo momento
Payload – imagem que está sendo enviada
Estrutura jason (forma estruturada de envio de dados, MUITO UTILIZADA)

Cabeçario muito simples, controle de 1 a 5 bites

SIMPLICIDADE PARA MANTER O PACOTE PEQUENO PARA NÃO SOBRECARREGAR A REDE É O PRINCIPAL DIFERENCIAL DO MQTT

==========

Onde usa:

Agricultura inteligente: controle da umidade do solo para confirmar a necessidade de irrigar ou não (sensores separados)
Casa inteligente: botões para ligar luzes, e todos os recursos tecnológicos
Monitoramento industrial: máquinas informam falhas e problemas, facilitando a resolução, como a medição de pressão (é perigoso)
Veículos conectados: rastreadores de localização funcionam a todo momento

QoS – Qualidade do serviço

QoS 0 --> Entrega única da mensagem, sem a confirmação (a mensagem é enviada independentemente se ela chegou ou não)

QoS 1 --> Garante pelo menos uma entrega confirmada para o tópico, mas as mensagens podem ser duplicadas

QoS 2 --> Garante que a mensagem seja enviada apenas 1 vez, sem se duplicar, porém é mais pesado que causa a sobrecarga (mais confiável)

Se a mensagem precisa ser enviada a todo momento, o QoS é mais indicado
Se precisa de um controle mais específico, o QoS 2 é mais indicado, caso precise registrar só 1 vez, como a confirmação de presença de um funcionário, ou ligar e desligar a luz)
==========

Persistência e sessões:

Broker tem que garantir o armazenamento da mensagem até que o cliente se reconecte e consuma essa mensagem
Falg retrain --> Garante que a mensagem só seja apagada quando for consumida por todos os assinantes, e isso é o que promove a persistência dos dados

Sessão - Armazena a última mensagem, capacidade de manter informações dentro do tópico enquanto for necessária

MQTT --> Não é criptografado, mas pode rodar por TLS, mas também autenticando usuário e senha ou controle de acesso por tópicos

Brokers MQTT:

Mosquito: de graça, dentro da própria máquina
HiveMQ: Pago, mais atrativo, mas capacidade
EMQX: Pago, alta performace
Outros: AWS IoT, Azure IoT, Google Cloud IoT (são em nuvem, nada local)

==========

Vantagens:
- Baixo consumo de energia e de dados
- Fácil de implementar
- Consegue implementar vários dispositivos dentro de um mesmo tópico (escalável)

Limitações:
- Só funciona com TCP/IP (nem tanto pois o TCP é o mais utilizado)
- Não tem criptografia (só tem mais segurança utilizando o TLS)
- É pequeno e simples para projetos/fluxos maiores (nesse caso HTTP é melhor)